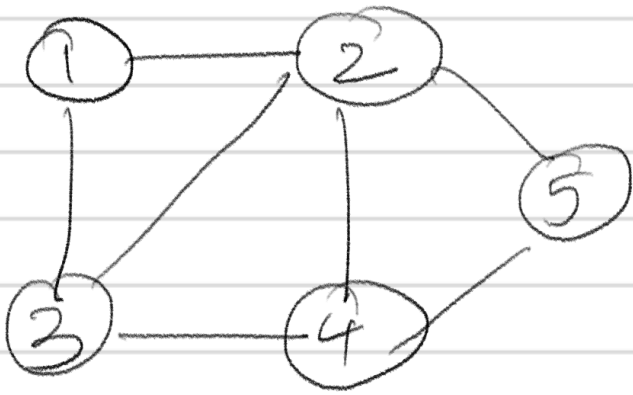


How to represent a graph



1) Adjacency list
array of lists

each member of the
array is a vertex, the
list contains all the vertices
adjacent to that vertex

1 → 2 → 3

2 → 1 → 3 → 4 → 5

3 → 1 → 2 → 4

4 → 3 → 2 → 5

5 → 2 → 4

2) Adjacency matrix

	1	2	3	4	5
1	0	1	1	0	0
2	1	0	1	1	1
3	1	1	0	1	0
4	0	1	1	0	1
5	0	1	0	1	0

For directions use 1, -1
in case of directed graphs

Directed graphs - edges have direction

undirected " - undirected edges

Breadth first search

- 1) Works on directed and undirected graphs.
- 2) Given a source vertex finds all the vertices that are reachable
- 3) Finds the shortest path from the source vertex to any vertex in the graph.
- 4) Called BFS because it explores all nodes at a certain level, before going to the next level

BFS

for all $v \in G$
 $v.color = white$
 $v.d = \infty$
 $v.p = NIL$

$O(V)$

source.d = 0
source.color = gray

Q.enqueue(source) first in first out)

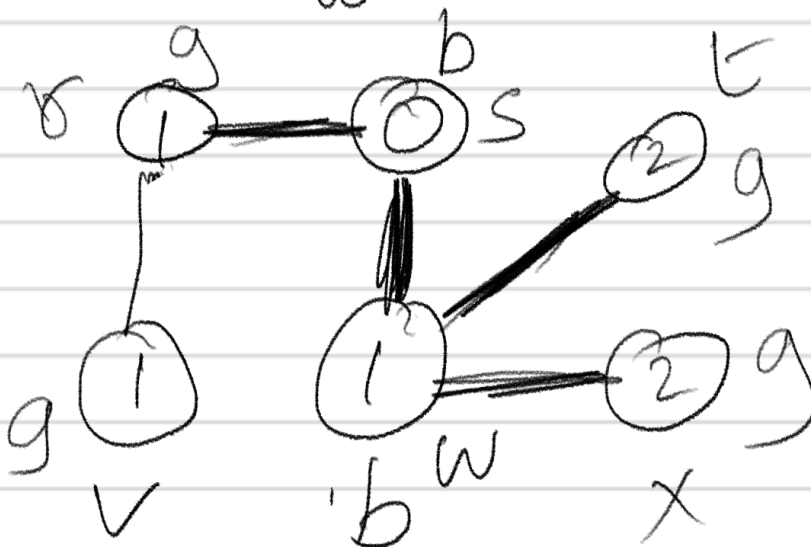
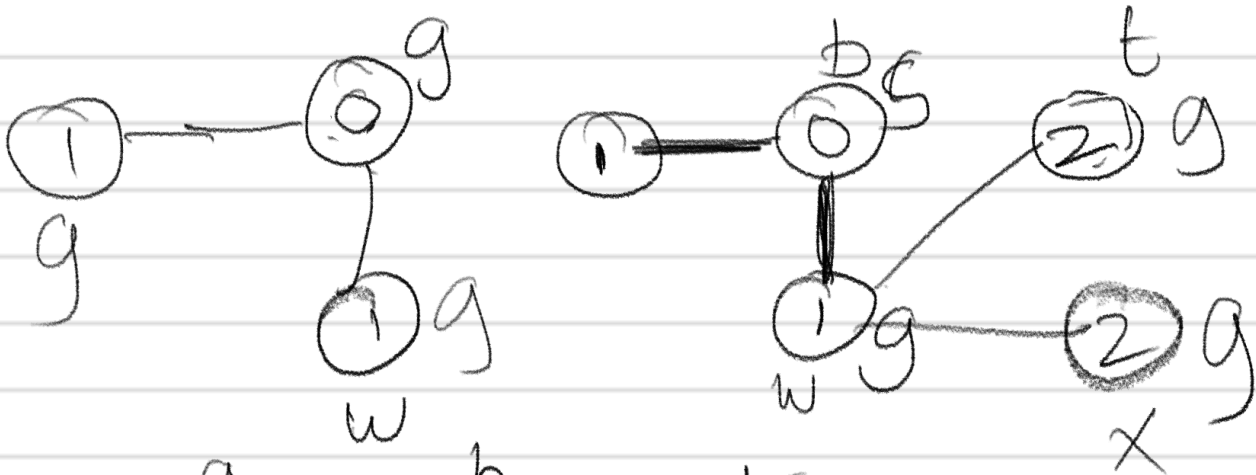
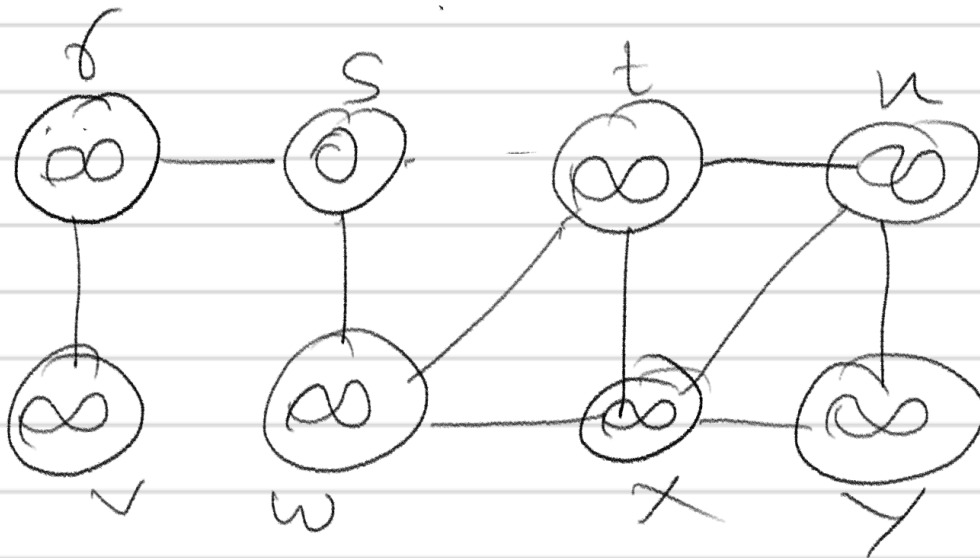
while (Q $\neq$ empty)

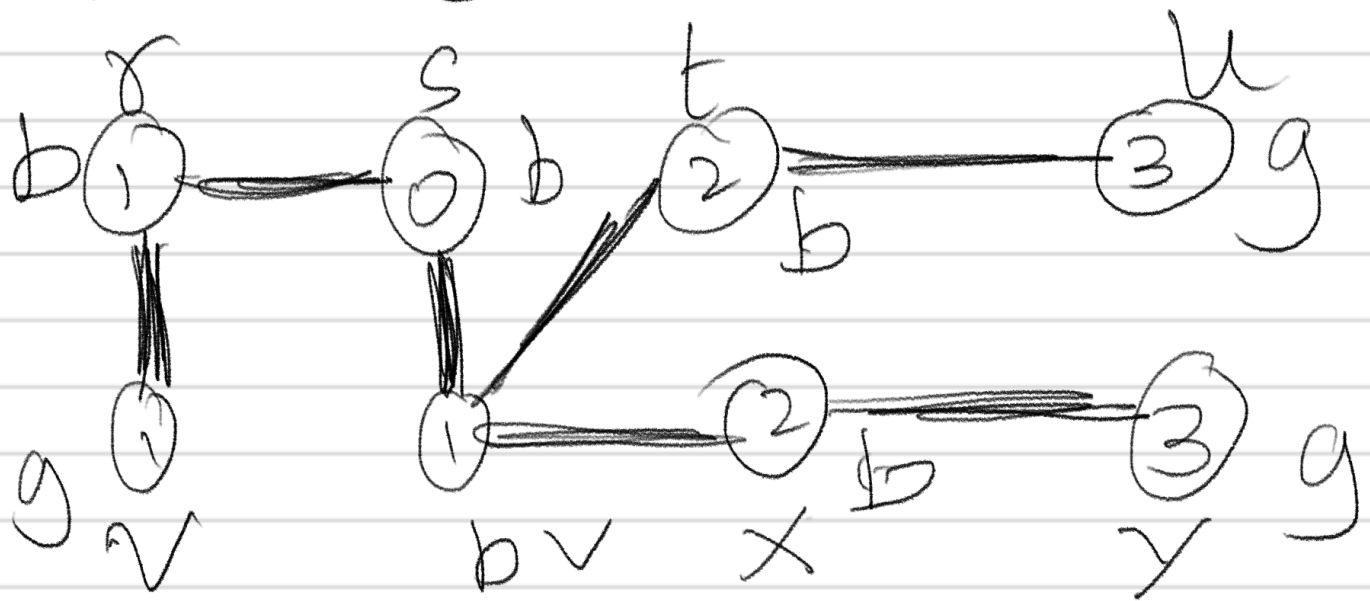
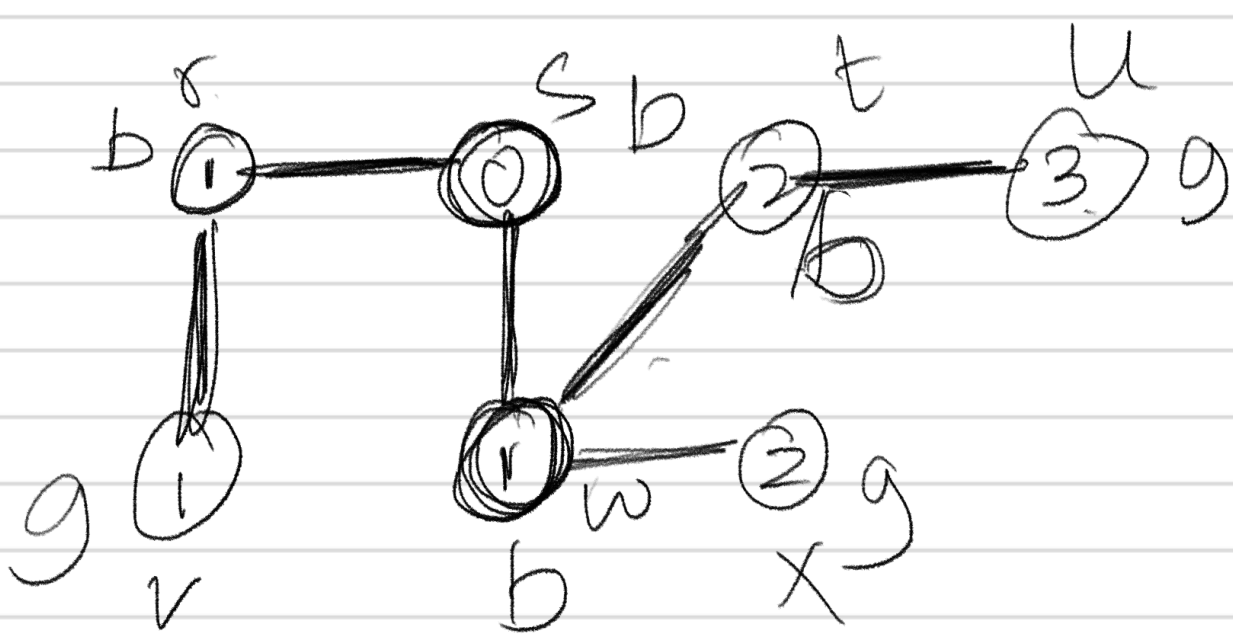
$O(E)$ $u = pop(Q)$
for all $v \in adj(u)$

```

if (v.color == white) {
    v.color = gray
    v.p = u
    v.d = u.d + 1
    enqueue(v)
}
u.color = black

```





v is black

x y become black

$$O(V + E)$$

Depth first search

- 1) searches from a vertex to the leaf backtracking to the previous to all its leaves and so on.
- 2) DFS grows as a forest of trees, if a source is given one tree, else when run on the whole graph multiple trees

3) Timestamping is possible when the graph is explored to find ancestor descendant relations

4) Tree edges from ancestor to descendant in the tree

5) Back edge $\rightarrow$ descendant to ancestor edge (loop)

6) Forward edge $\rightarrow$ ancestor to descendant but does not belong to the tree.

6) cross edges are other edges in one trees between unrelated

vertices

DFS (G)

for each $v \in G$

$v.f = -1$

$v.p = \text{NIL}$

$v.\text{color} = \text{white}$

for each $u \in G$

if ($u.\text{color} == \text{white}$)

DFS (G, u)

DFS (G, u)

$u.\text{color} = \text{gray}$

for all $v \in \text{adj}(u)$

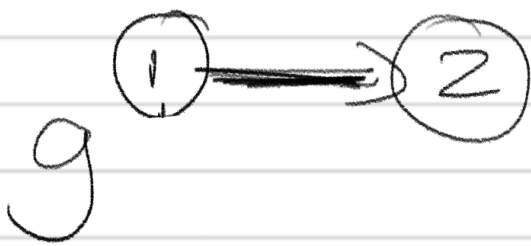
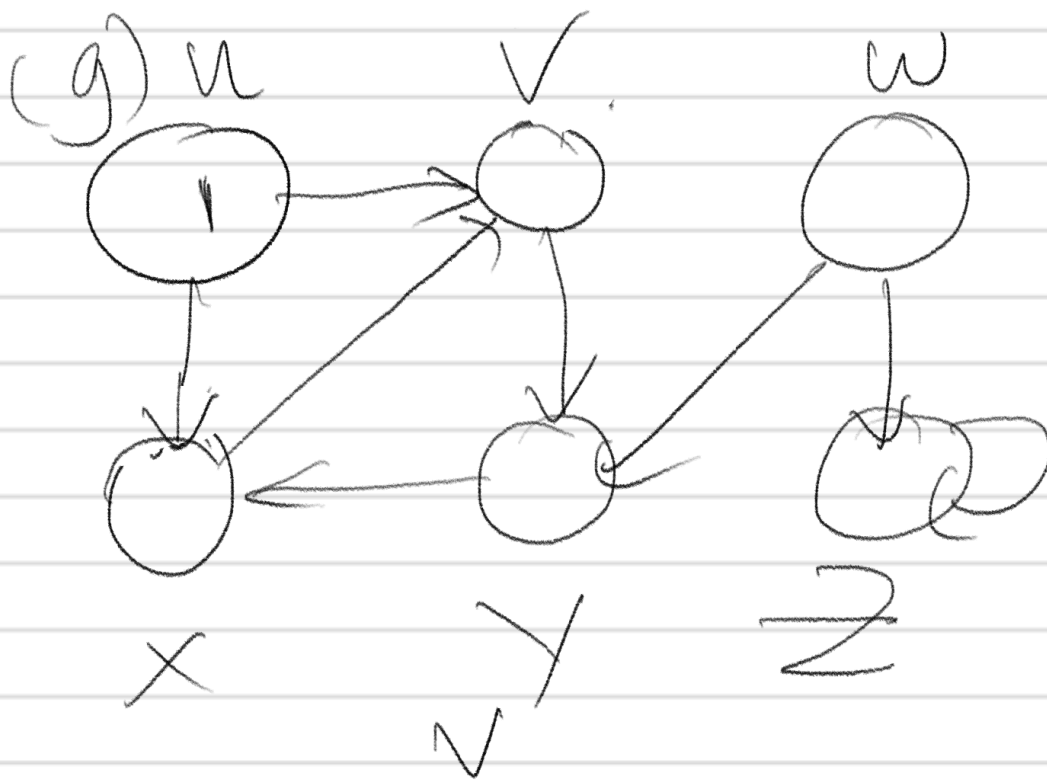
if $C_v, \text{color} == \text{white}$

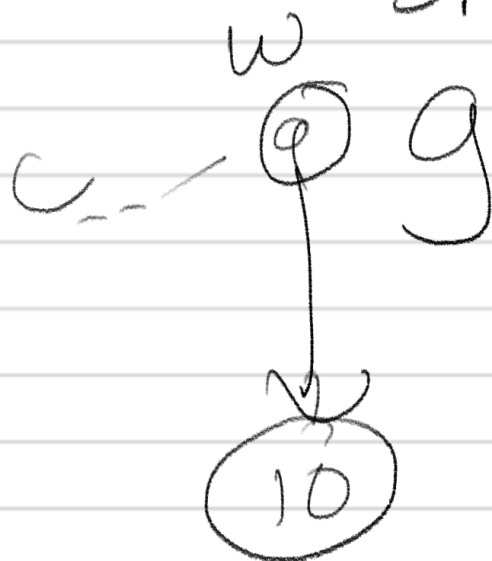
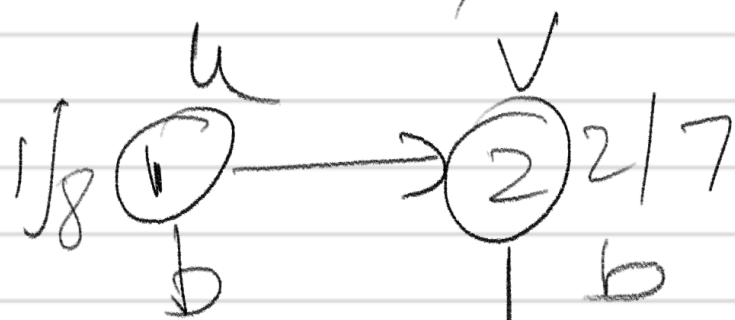
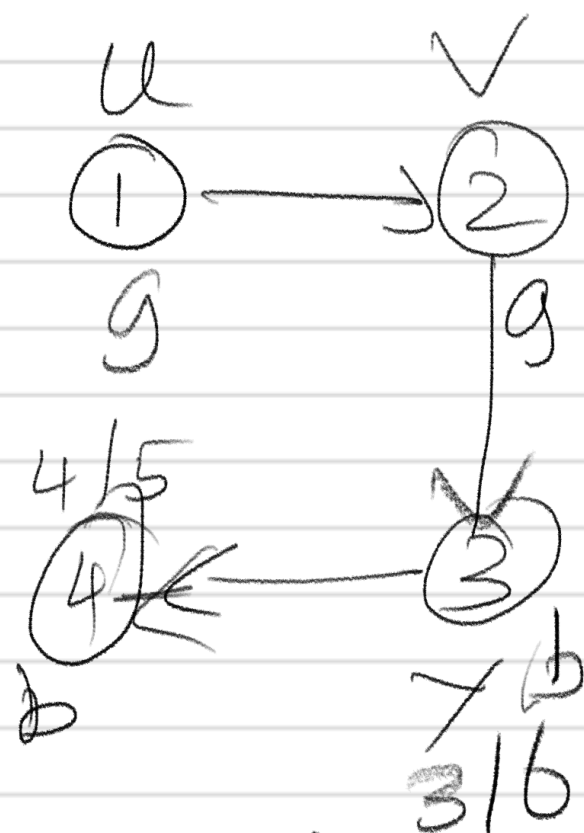
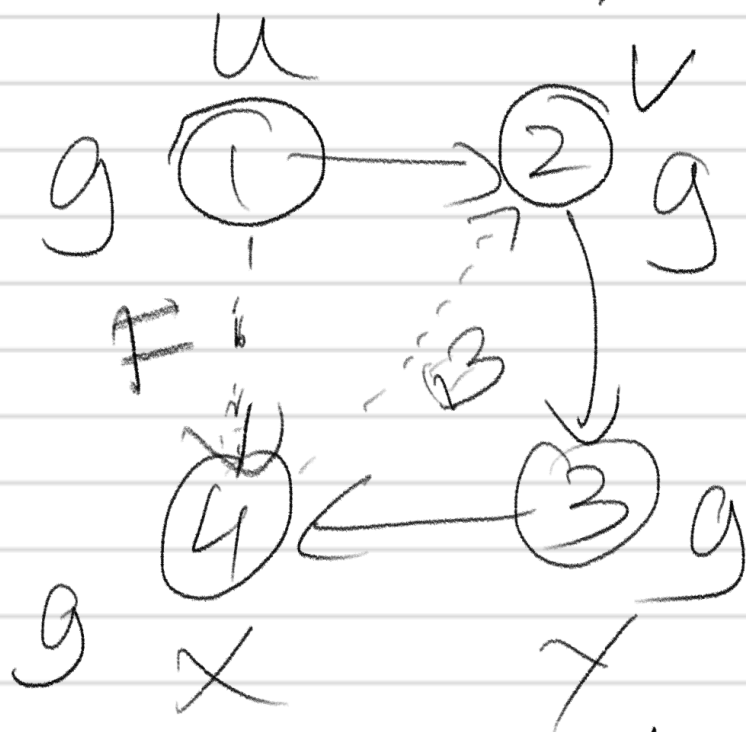
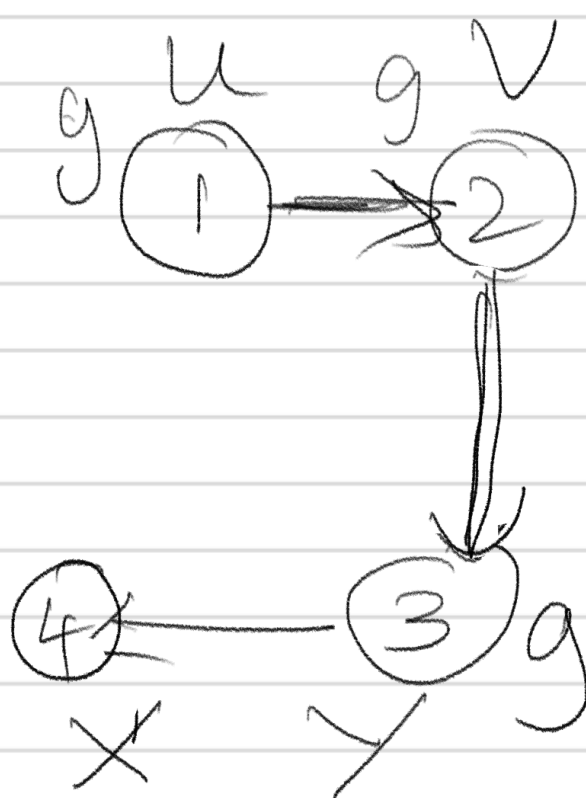
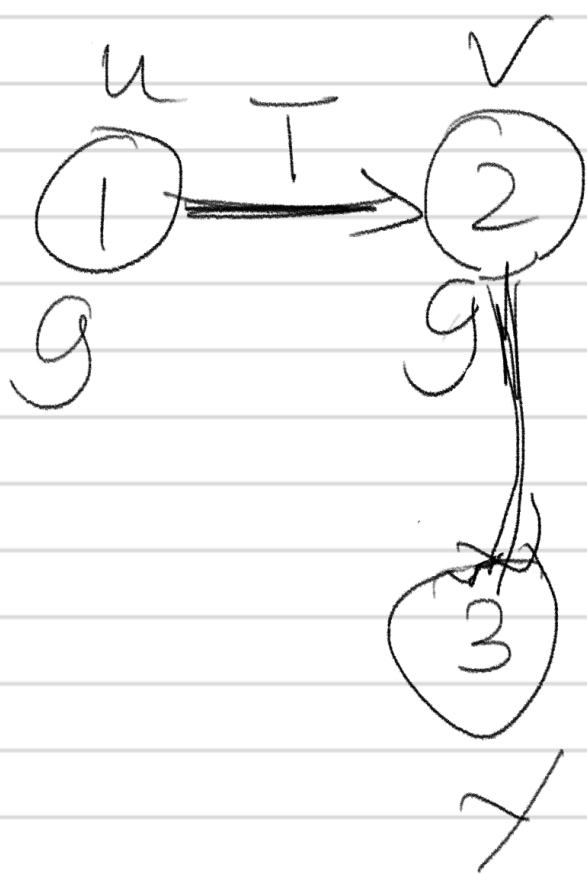
$v.p = u$

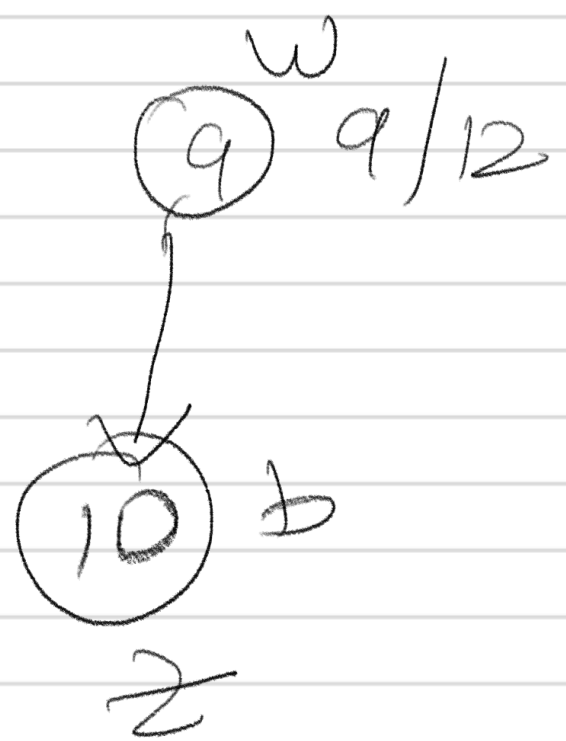
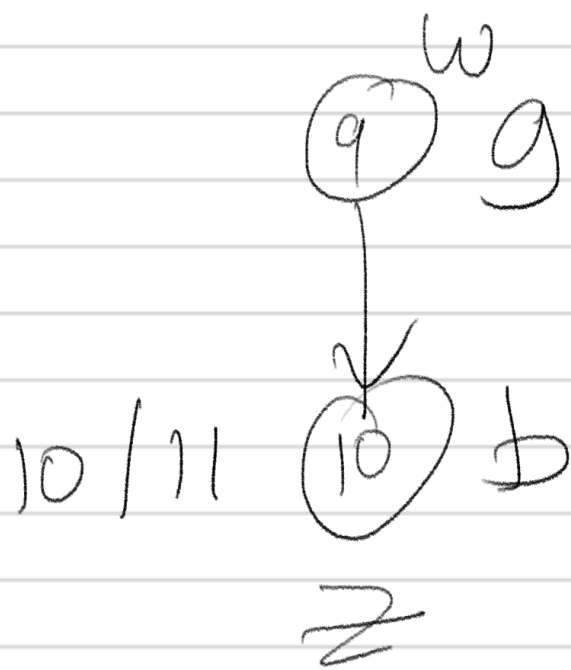
$\text{DFS}(G, v)$

$u.\text{color} = \text{black}$

$u.f = \text{cur_time}$







Topo sorting
Run DFS
as each vertex
add to front of
list

Minimum spanning Tree

An acyclic set of vertices such that vertices are connected and the sum of weights of the edges is minimized

Prim's

for all $v \in G$ $O(V)$
 $v.p = \text{NIL}$

$v.\text{key} = \infty$

source.key = 0

$Q.\text{enqueue}(G)$ (priority)
while ($Q \neq \text{empty}$) Q

$u = \min(Q)$

for all $v \in \text{adj}(u)$

if ($v \in Q$ && $w(u,v) < v.\text{key}$)

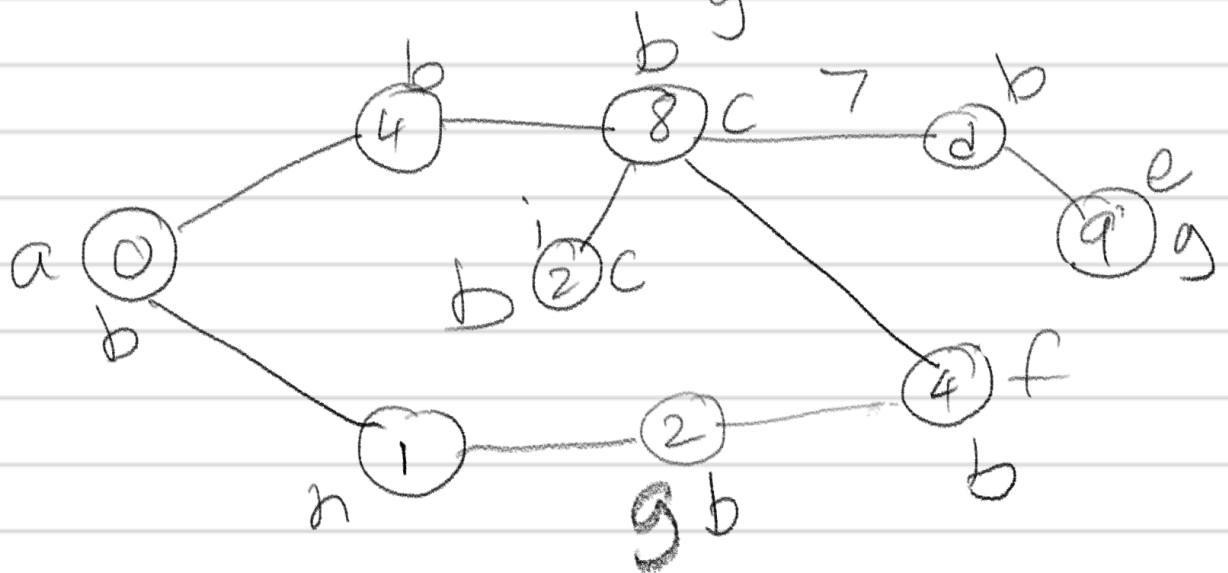
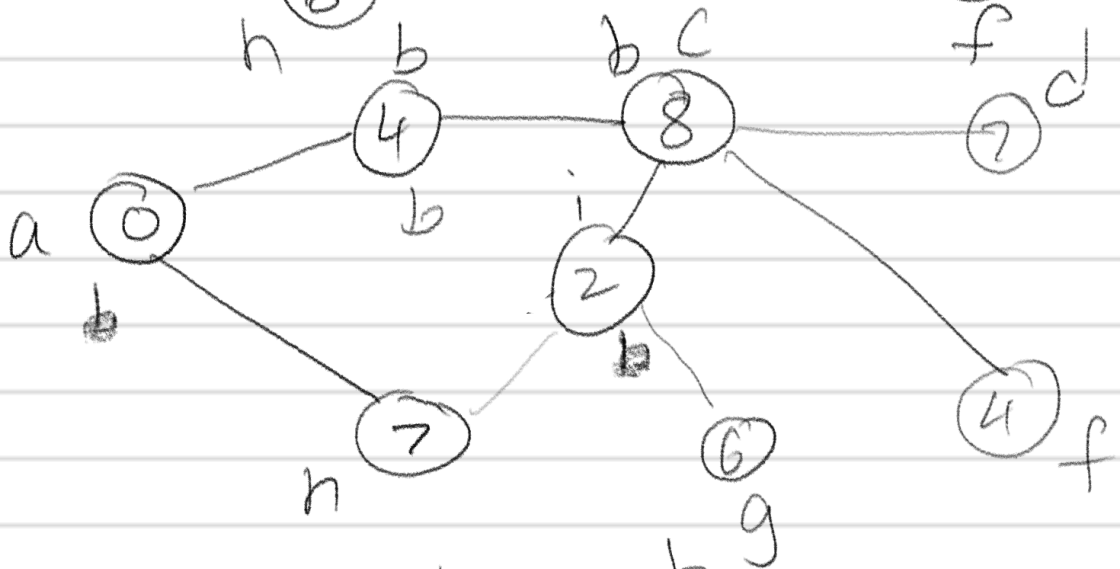
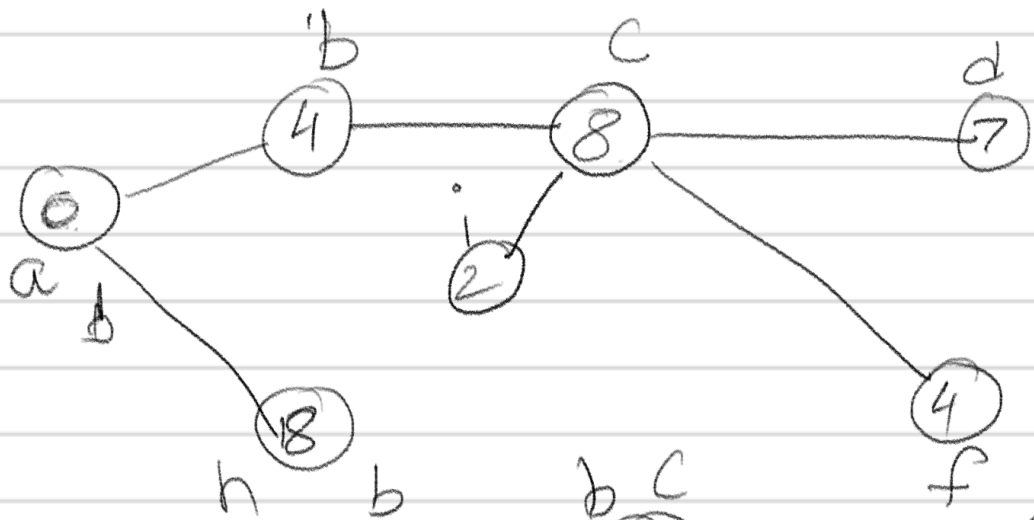
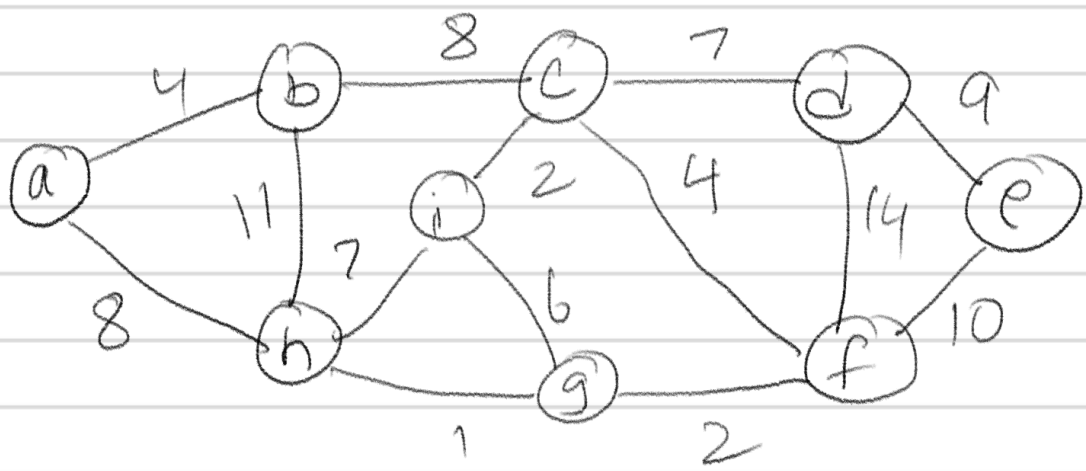
$v.\text{key} = w(u,v)$

$v.p = u$

$O(V \lg V + E \lg V)$

$V \lg V$

$E \lg V$



Dijkstra's algorithm

Single source shortest path
for a weighted directed graph
with positive edge weights
for all $v \in G$

$$v.d = \infty \quad v.p = \text{NIL}$$

$$s.d = 0$$

Q. Enqueue(G)

while (Q $\neq$ empty)

u = Q.extractmin()

S = S $\cup$ {u}

for all $v \in \text{adj}(u)$

if ($w(u, v) + u.\text{key} < v.\text{key}$)

v.p = u

v.key = $w(u, v) + u.\text{key}$

$\in \lg v$